

Rapport: NoshNarc

Gruppenummer:

Medlemmer:

Jonas Hellevik

Odd E. Helgøy

Maria Tysseland Kristoffersen

Dato: 29/10-2025

1: PROSJEKTBEKRIVELSE	2
2: DATA	3
3: MODELLERING	4
4: DEPLOYMENT	5
5: REFERANSER/KILDER	6

1: PROSJEKTBEKRIVELSE

Er det mulig å forutsige hvilken hurtigmatkjede et gitt hurtigmatsprodukt kommer fra?

- Argumenter for ML:

- Mye data å gå gjennom som er satt en felles standard på som ikke er bestemt av bedriftene selv. Dette gjør at vi kan sammenligne til tross for undersøkelse hos forskjellige bedrifter. Informasjonen av interesse i dette tilfellet er næringsinnholdet til artiklene.

Mange datapunkter, som er relevante for å kunne gjøre en forutsigelse med tolererbar nøyaktighet - ikke mulig for et menneske å analysere for hånd siden det er flerdimensjonalt.

- Hvordan man kan bruke løsningen:

- For morroskyld/lærdom hvis man har interesse for maskinlære
- Kan potensielt gi innsikt i hvordan hurtigmatskjeder er strukturert
- Mulighet for å prøve med egendefinert input for å la modellen forsøke å gjette

Finnes det tilsvarende løsning(er) i dag?

Det finnes løsninger som ved å se på et bilde av et produkt eller får navnet på et produkt kan fortelle næringsinnholdet til produktet, det vi gjør er jo det motsatte. (Vi har kommet på ideen selv etter å surfe gjennom datasett. Dermed er det en større utfordring å finne en eksisterende løsning. Det finnes prosjekter og applikasjoner som ligner, men ingenting vi har funnet er helt likt.

Er det mulig å gjøre dette uten ML?

Dette ville vært en utrolig slitsom prosess å gjøre uten maskinlæring, da man måtte pugget eller lest av næringsinnholdet tilsvarende treningssettet, det finnes ingen passende algoritmer man kunne tatt i bruk. Man måtte lett etter sammenhenger basert på næringsinnholdet, og ofte er variasjonene ganske små. (Det hadde vært mer [realistisk](#) for et menneske å identifisert ut i fra bildet av en artikkel med eventuelle merkevare logoer usynliggjort)

Businessrelevans:

- Ikke relevant, (for oss som har skrevet oppgaven) ettersom det er ikke en særlig komplisert oppgave vi har løst. Det kan godt være mulig at en eller flere av disse hurtigmatskjedene har gjort lignende analyser internt, ettersom hurtigmat er verdens mest overkonstruerte mat man kan kjøpe - spesielt hvis det kommer fra gigantene som McD, BK etc. Men for privatpersoner har egentlig ikke prosjektet noen praktiske formål eller bruksområder annet enn for en eventuell introduksjon til maskinlæring.

Metrikker:

- Accuracy, recall og f1 -> Indikasjon på ytelsen til modellen
- Confusion matrix -> Viser antall ganger modellen forutså feil som blir en indikasjon på ytelse
- Feature importance -> Viser hvilke features som hadde mest uttelling for modellen gitt i prosent, brukes for å justere/optimalisere modellen.
- Training history -> Lagring av seed, treningsandel, modellvalg og nøyaktighet for å kunne sammenligne ulike treningsoppsett.

Brukertilfredsstillelse - Vi har prøvd å lage mange funksjoner og visualiseringer i webapplikasjonen, men kan alltid gjøre små endringer basert på feedback i etterkant, dette er bare et gratisprosjekt som tenkes å la stå som et offentlig prosjekt.

2: DATA

Vi fant et dataset med et hav av artikler fra 6 forskjellige hurtigmatskjeder. Datasettet inneholder all tilgjengelig informasjon om næringsinnholdet til artiklene, for eksempel: kalorier, fett, salt, sukker, karbohydrater, proteiner etc. All informasjon er standardisert til porsjonsstørrelse.

For skalering:

For å skalere prosjektet kan man utvide datasettet med nye menyprodukter samt flere hurtigmatskjeder. Prosjektet bruker Unsupervised Learning, så labels er ikke nødvendig. Det er heller ikke behov for noe personvernshensyn eller etiske betraktninger ettersom datasettet inneholder ingen personlige opplysninger, og vi lagrer også ingen slike opplysninger gjennom webapplikasjonen.

For rensing av data:

- Rensing med GPT-5: Gav prompten 'Can you validate if this file follows its own format correctly?' og ga selve datasettet fra Kaggle i samme prompt som en fil. Så valgte GPT-5 å produsere filen `fast_food_cleaned.csv` på eget initiativ. Det gjorde det enkelt å kunne mate datasettet inn i ML-modellen uten å måtte gjøre så mye mer formatering i selve koden (inni [preprocessing.py](#))

For å lage trenings- og testsettet: (Preprocessing)

- Satte av 'Company' columnen til å være vår Y variabel (target)
- Fjernet 'Item' columnen helt siden den består av navn på produktene som kan enten røpe for mye hvis de inneholder navnet på selskapet i selve produktnavnet, ellers så er navnene overflødige og har ikke noen betydning for modellen.
- Utenom 'Company' og 'Item' så er resten av colonnene inkludert i X variabelen (features), først etter å ha konvertert alt til numeriske verdier.

(Satte av 30% av datasettet til testing etter å ha prøvd oss litt frem - var opprinnelig satt til 20% som standardverdi)

3: MODELLERING

To gjeldende modeller:

- RandomForrestClassifier
- HistGradientBoostingClassifier

Modellene er direkte utskiftbare med hverandre i koden, og har lignende ytelse etter litt testing. Det er også da laget støtte for å kunne trene modellen ved kjøretid med egendefinerte verdier (prosentandel-treningsett og seed), med mulighet til å velge en av de to modellene.

Hvorfor valgte vi disse modellene kontra for eksempel LinearRegression? -> Brukte GPT-5 for forslag for å ha et startpunkt, og det viste seg at RandomForrestClassifier traff bedre enn forventet (85%~) og da var vi allerede ganske fornøyd med det, og fant

HistGradientBoostingClassifier siden den er også i sklearn.ensemble)

Modellene ble også valgt med hensyn til at de har medført støtte til NaN verdier, noe som gjorde det mye enklere å bearbeide datasettet.

Baseline:

- Ha en sett seed (default=42) slik at det er mulig å gjenskape forhold for å kunne lett sammenligne endringer av evt modell eller prosentandel-treningsett. Vi har og lagt inn lagring av historikken for å sammenligne med tidligere resultater.

Hvordan undersøkes feil prediksjoner og feature importance:

Confusion matrix viser feil prediksjoner, og den viser tydelig samsvar med accuracy/recall - samt at vi ser at de gangene modellen tar feil så er det som regel som forventet. For eksempel så forveksles Burger King med McDonalds. Det kan være fordi det er begge burgerkjeder, og har dermed lignende menyer, men interessant nok så går ikke forvekslingen like mye andre veien.

Feature importance (FI):

Cals from fats, weight watcher points -> mulige 'forgiftede' metrikker som underspiller viktigheten av sugar, carbs, calories, protein og total fat.)

Bør undersøke å fjerne disse to kolumnene for å se om vi får en ny feature importance som blir litt som vi forventer:

Forventning:

- Cals from fats(nr 1 i FI) -> (one to one, 100% vektoverføring) egentlig bare Total fats*9, så Total Fats burde bli lagt mye mer vekt på.
- WWP (nr 2 i FI) -> (one to four, 25% vektfordeling -> blir delt på fire) burde føre til en mer jevn fordeling av ny FI siden denne bare bruker de andre dataene (kalorier, sukker, mettet fett og proteiner) for å gi en "score".

Forutsigelsen da er at Total Fats ender opp med størst vekt i en ny FI, ellers så er det en noe lunde jevn fordeling av resten. Ved jevnere fordeling, og i tillegg til å unngå gjenbruk av features som er basert på andre features, så kan man se for seg at ytelsen kan bli bedre som resultat.

4: DEPLOYMENT

Deployment:

- Streamlit webapplikasjon som kan tas i bruk av hvem som helst, offentlig repository på [GitHub](#) med lenke til [webapplikasjonen](#) vedlagt i [readme.md](#) filen.
- Vedlikehold er minimalt siden det er mer et 'passion-prosjekt' som er offentlig tilgjengelig og ikke tatt betalt for. Prosjektet anses omtrentlig til å være ferdig, med de fleste ønskede funksjoner ferdigimplementert. Det er bare små tweaks som er gjeldende, evt tillate å tweake max depth på modellen i webapplikasjonen fremfor, modellen bruker hele datasettet på å bygge sine trær ellers.
- Bruk: Kan være et greit prosjekt for studenter som nettopp har begynt med ML selv for å ha som et eksempel der det er mulighet å kjøre modellene vi har valgt, og lett få litt visuell feedback med en gang - uten å måtte skrive kode selv.

Instrukser for bruk av webapplikasjon:

Modeller er forhåndskonfigurert ved start, så man trenger ikke å trene modellen hvis man bare ønsker å se på eksemplene eller teste den.

Tren en modell på egenhånd:



- Seed: Kan settes inn manuelt eller trykke på 'Random seed' for en tilfeldig generert seed.
- Test size (%): Slider fra 1 til 99 for å velge hvor stor andel av datasettet legges til side som test set.
- Model: Dropdown valgmeny mellom to modeller.
- Retrain: Trykk for å trene modellen på nytt.

Etter å ha valgt en konfigurasjon og trent modellen, så kan man bla ned for en oversikt over presisjon, hukommelse og f1-score, samt confusion matrix, feature importance, og historikk over tidligere modellkonfigurasjoner.

Nedest på siden kan man teste modellens evne til å gjette korrekt ved enten legge inn næringsinformasjon selv, eller velge noe fra datasettet som ble brukt til å trene modellen i utgangspunktet.

5: REFERANSER/KILDER

Notat i: (i henhold til valg av ML modell)

Feedback fra feilmelding:

ValueError: Input X contains NaN. LinearRegression does not accept missing values encoded as NaN natively. For supervised learning, you might want to consider `sklearn.ensemble.HistGradientBoostingClassifier` and `Regressor` which accept missing values encoded as NaNs natively. Alternatively, it is possible to preprocess the data, for instance by using an imputer transformer in a pipeline or drop samples with missing values.

LogisticRegression: samme feilmelding som LinearRegression

HistGradientBoostingClassifier: i bruk

RandomForrestClassifier: i bruk

Notat ii:

Etter mange tester: 30% test size virker til å være "sweet spot" for høyest accuracy for begge modellene vi har valgt å inkludere i prosjektet.

Deployed webapplikasjon av prosjektet:

<https://h669985-fastfoodpredicter-app-iuuots.streamlit.app/>

Offentliggjort GitHub repository av prosjektet:

<https://github.com/h669985/FastFoodPredicter>

Dataset:

<https://www.kaggle.com/datasets/tan5577/nutritonal-fast-food-dataset?select=FastFoodNutritionMenuV3.csv>

AI bruksærklæring: ChatGPT-5 ble brukt for å generere filen [fast_food_cleaned.csv](#), [visualize_result.py](#) og [app.py](#) (streamlit webapplikasjonen)

Kilde for hvordan vi fikk filen 'fast_food_cleaned.csv':

Promten som ble gitt:

['Can you validate if this file follows its own format correctly?' og selve datasettet fra kaggle i samme prompt lastet opp som fil utenom tekstfeltet

GPT-5 sitt svar:

[I took a close look at the file you uploaded and ran a structural validation. Here's the verdict:

What's correct

- It's valid CSV: consistent **comma delimiter**, and every row has the same **14 columns**.
- There is a header row, with **no duplicate or blank column names**.
- Encoding works fine with **UTF-8**.

Issues I found

- Several header names contain **embedded newlines** and extra spaces (e.g., **Total Fat\n(g)**, **Sodium \n(mg)**, **Weight Watchers\nPnts**). That's unusual and can break downstream tools.
- A number of columns that should be numeric contain some **non-numeric values** (likely placeholders like dashes/strings). Counts flagged during coercion:
 - **Calories**: 14 rows non-numeric
 - **Cholesterol\n(mg)**: 28 rows
 - **Sodium \n(mg)**: 15 rows
 - **Carbs\n(g)**: 13 rows
 - and similar smaller counts across fat/protein columns
(Text columns like **Company** and **Item** are, of course, non-numeric —

that's expected.)

So: structurally the CSV is consistent, but the **column headers contain newlines**, and there are **some non-numeric entries in numeric fields**. If your “own format” expects clean unit labels in headers and purely numeric nutrient fields, then strictly speaking it does **not** fully adhere.

What I generated for you

- A **preview of the first 10 rows** is displayed above in the workspace so you can eyeball it.
- A concise **validation report** with the key checks was saved for you:
Download the validation report (var en lenke)
- I also produced a **cleaned CSV** that:
 - Normalizes headers by removing newlines and extra spaces (e.g., **Total Fat (g)**, **Sodium (mg)**, etc.)
 - Coerces numeric columns to numbers (invalid entries become **NaN**)
- **Download the cleaned file (var en lenke)**

Summary

- **Pass** on core CSV structure (delimiter, row length consistency, header present).
- **Fix needed** on header formatting (newlines/whitespace) and a handful of **non-numeric cells** in numeric columns.

If you want, I can also:

- List the **exact rows** that failed numeric conversion for each column,
- Replace common placeholders (e.g., **–**, **varies**) with **NaN**,
- Or enforce **value ranges** (no negatives, realistic nutrient bounds) for a stricter spec check.]

(Bilde av preview som GPT-5 nevnte i sitt svar)

CSV Preview (First 10 Rows)					 	
	Company	Item	Calories	Calories from Fat		
6	McDonald's	Double Quarter Pounder® with Cheese	740	380		
7	McDonald's	Big Mac®	540	260		
8	McDonald's	Big N' Tasty®	460	220		
9	McDonald's	Big N' Tasty® with Cheese	510	250		
10	McDonald's	Angus Bacon & Cheese	790	350		

CSV Preview (First 10 Rows)					 	
	Company	Item	Calories	Calories from Fat		
1	McDonald's	Hamburger	250	80		
2	McDonald's	Cheeseburger	300	110		
3	McDonald's	Double Cheeseburger	440	210		
4	McDonald's	McDouble	390	170		
5	McDonald's	Quarter Pounder® with Cheese	510	230		